

Networking Assignment

1. Set up two focal VMs and assign IPs from the same network range eg.172.16.0.0 /24 and check if ping works.

Step 1.1: ***multipass set local.bridged-network=en0***

What it does: Sets the **default network interface** on the host machine (**en0** in this case) that Multipass should **bridge** its VMs to.

Why: This is essential for the VMs to get IPs from the **same network** as the host, allowing them to communicate directly with each other and, often, the outside world. When a computer needs to send or receive data from a network, it uses one of these interfaces.

Step 1.2: ***multipass launch focal --name vm1 --bridged***

What it does: Launches a new **Ubuntu Focal (20.04 LTS)** VM, names it **vm1**, and instructs Multipass to connect it using the **bridged network** configuration set in Step 1.1.

Step 1.3: ***multipass launch focal --name vm2 --bridged***

Step 1.4: ***multipass shell vm1***

Step 1.5: ***sudo vim /etc/netplan/50-cloud-init.yaml***

Netplan is the standard network configuration tool in modern Ubuntu systems. This file is modified to **switch the VM from DHCP to a static IP** configuration (**172.16.0.10/24**) on the **enp0s2** interface.

```
None
network:
  version: 2
  renderer: networkd
  ethernets:
```

```
enp0s2: // real world network
  dhcp4: false
  addresses: [172.16.0.10/24]
  routes:
    - to: default
      via: 172.16.0.1
  nameservers:
    addresses: [8.8.8.8, 8.8.4.4] // google dns
enp0s1: // private network
  dhcp4: true
  optional: true
```

Step 1.6: *sudo netplan try*

What it does: Attempts to apply the new network configuration from the Netplan file and **waits for user confirmation**. If connectivity is lost or times out, it reverts the changes

Why: This is a safety feature. It allows you to test the new configuration before making it permanent, preventing you from accidentally locking yourself out of the VM if the settings are wrong.

Step 1.7: *sudo netplan apply*

What it does: **Permanently applies** the currently configured network settings (the static IP configuration) without the safety check of `netplan try`.

Why: Once you've confirmed the configuration works (usually after a successful `netplan try`), this command makes the change definitive.

Step 1.8: *multipass shell vm2*

Step 1.9: *sudo vim /etc/netplan/50-cloud-init.yaml*

What it does: Opens the **Netplan configuration file** inside `vm2` for editing. The file is edited with a similar YAML configuration but with the IP address **172.16.0.20/24**.

Why: This sets a **different, unique static IP** for `vm2` within the same network range (`172.16.0.0/24`) as `vm1`, which is necessary for the two machines to communicate.

```
None
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s2:
      dhcp4: false
      addresses: [172.16.0.20/24]
      routes:
        - to: default
          via: 172.16.0.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
    enp0s1:
      dhcp4: true
      optional: true
```

Step 1.10: *sudo netplan try*

Step 1.11: *sudo netplan apply*

Step 1.12: *multipass exec vm1 -- ping -c 4 172.16.0.20*

What it does: Executes the `ping` command **remotely** inside `vm1`. The command sends **4 ICMP packets** (`-c 4`) from `vm1` to the IP address of `vm2` (`172.16.0.20`).

Why: This is the final and most important step: it tests **connectivity** between the two virtual machines. A successful ping confirms that both VMs are correctly configured on the same bridged network and can communicate using their assigned static IPs.

```
vm1                Running                192.168.64.46      Ubuntu 20.04 LTS
                  172.16.0.10
vm2                Running                192.168.64.47      Ubuntu 20.04 LTS
                  172.16.0.20
vm3                Running                192.168.64.40      Ubuntu 20.04 LTS
[shambhavi.intern@PP-R7MV533L02 ~ % multipass exec vm1 -- ping -c 4 172.16.0.20
PING 172.16.0.20 (172.16.0.20) 56(84) bytes of data.
64 bytes from 172.16.0.20: icmp_seq=1 ttl=64 time=0.732 ms
64 bytes from 172.16.0.20: icmp_seq=2 ttl=64 time=3.40 ms
64 bytes from 172.16.0.20: icmp_seq=3 ttl=64 time=14.1 ms
64 bytes from 172.16.0.20: icmp_seq=4 ttl=64 time=4.82 ms

--- 172.16.0.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3174ms
rtt min/avg/max/mdev = 0.732/5.752/14.063/5.017 ms
shambhavi.intern@PP-R7MV533L02 ~ % █
```

2. Add another focal VM to the setup and assign connectivity in the following order where VM2 acts like a router. VM1 <->VM2<->VM3.

VM1 should be from 172.16.0.0 /24 and VM3 should be from 192.168.0.0 /24. Should be done using static routing.

Before these steps, we would need to launch the third VM using a command like: `multipass launch focal --name vm3 --bridged`

Step 2.1: ***multipass shell vm2***

Step 2.2: ***sudo vim /etc/netplan/50-cloud-init.yaml***

What it does: Opens the Netplan configuration file for editing.

Why: The core of the router configuration is done here. The new configuration adds the second network's gateway address (192.168.0.1/24) to the same network interface (enp0s2). It assigns **two different IP addresses** from two different subnets to the **single bridged interface** (enp0s2). 172.16.0.20 serves as the gateway for vm1, and 192.168.0.1 serves as the gateway for vm3. This is how vm2 sits on both networks simultaneously.

```
None
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s2:
      dhcp4: false
      # Assign both gateway IPs to the single bridged interface
      addresses: [172.16.0.20/24, 192.168.0.1/24]
    enp0s1:
      dhcp4: true
      optional: true
```

Step 2.3: ***sudo netplan try***

Step 2.4: ***sudo netplan apply***

Step 2.5: ***exit***

Step 2.6: ***multipass shell vm3***

Step 2.7: ***sudo vim /etc/netplan/50-cloud-init.yaml***

What it does: Opens the Netplan configuration file for editing.

Why: Configures **vm3** to be a client on the new network segment. Assigns a **static IP** from the new subnet (192.168.0.0/24) to **vm3**. This address is unique to **vm3** and allows it to communicate with the 192.168.0.1 address on **vm2**. Sets the **default gateway** to the IP address that **vm2** is using on the 192.168.0.0/24 subnet. This is the crucial routing step: it tells **vm3** that to reach *any other network* (including 172.16.0.0/24), it **must send traffic to vm2** first.

None

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s2:
      dhcp4: false
      addresses: [192.168.0.2/24]
      routes:
        - to: default
          via: 192.168.0.1 # <-- Must point to VM2's 192.168.0.1 address
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
    enp0s1:
      dhcp4: true
      optional: true
```

Step 2.8: *sudo netplan try*

Step 2.9: *sudo netplan apply*

Step 2.10: *exit*

Step 2.11: *multipass exec vm1 -- ping -c 4 192.168.0.2*

What it does: Executes the `ping` command remotely inside `vm1`, attempting to reach `vm3`'s address (`192.168.0.2`) on the **other network**.

Why: This tests the path: **VM1** sends traffic to its default gateway (`172.16.0.20` on VM2), **VM2** forwards it across the networks to `192.168.0.2` on VM3.

Step 2.12: *multipass exec vm3 -- ping -c 4 172.16.0.10*

What it does: Executes the `ping` command remotely inside `vm3`, attempting to reach `vm1`'s address (`172.16.0.10`) on the **other network**.

Why: This tests the reverse path: **VM3** sends traffic to its default gateway (`192.168.0.1` on VM2), and **VM2** routes it to `172.16.0.10` on VM1.

```
shambhavi.intern@PP-R7MV533L02 ~ % multipass exec vm1 -- ping -c 4 192.168.0.2
PING 192.168.0.2 (192.168.0.2) 56(84) bytes of data.
From 172.16.0.20 icmp_seq=1 Redirect Host(New nexthop: 2.0.168.192)
64 bytes from 192.168.0.2: icmp_seq=1 ttl=63 time=1.26 ms
From 172.16.0.20 icmp_seq=2 Redirect Host(New nexthop: 2.0.168.192)
64 bytes from 192.168.0.2: icmp_seq=2 ttl=64 time=10.2 ms

--- 192.168.0.2 ping statistics ---
2 packets transmitted, 2 received, +2 errors, 0% packet loss, time 1040ms
rtt min/avg/max/mdev = 1.264/5.751/10.239/4.487 ms
shambhavi.intern@PP-R7MV533L02 ~ % multipass exec vm3 -- ping -c 4 172.16.0.10
PING 172.16.0.10 (172.16.0.10) 56(84) bytes of data.
64 bytes from 172.16.0.10: icmp_seq=1 ttl=64 time=0.365 ms
64 bytes from 172.16.0.10: icmp_seq=2 ttl=64 time=6.10 ms
64 bytes from 172.16.0.10: icmp_seq=3 ttl=64 time=2.65 ms
64 bytes from 172.16.0.10: icmp_seq=4 ttl=64 time=1.68 ms

--- 172.16.0.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3158ms
rtt min/avg/max/mdev = 0.365/2.700/6.100/2.124 ms
shambhavi.intern@PP-R7MV533L02 ~ %
```

3. Assign IPs to loopback (lo interface) and recreate #1 and #2 via BGP using FRR. VM1 lo: 10.1.1.1/32, VM2 lo: 10.1.1.2/32, VM3 lo: 10.1.1.3/32.

Ping should work between the loopback addresses. The network range for BGP IPs is 172.16.0.0/24 for both the scenarios.

To successfully transition to BGP dynamic routing (Task 3), we **must remove these static route definitions** from both VMs. If we don't, the VMs will ignore the BGP-learned routes and continue sending all non-local traffic to the old, static gateway, which interferes with dynamic routing.

- **BGP (Border Gateway Protocol):**
 - **Definition:** BGP is a sophisticated **dynamic routing protocol** used primarily to exchange routing information between large, separate networks (like those owned by ISPs or major corporations) across the Internet.
 - **Analogy:** It acts like the **global GPS of the Internet**, helping routers decide the best path for data to travel from one continent or network to another.
- **FRR (Free Range Routing):**
 - **Definition:** FRR is the **"brain" you install on a Linux computer to turn it into a high-performance router**. It provides the tools to let the computer talk to other routers, learn about network changes, and automatically find detours if a connection goes down.
 - **Analogy:** If BGP is the language of routing, **FRR is the translator or software** that lets your basic virtual machine speak and understand that language.
- **Loopback IP:**
 - **Definition:** A specific, fixed IP address (like 10.1.1.1/32) assigned to the loopback interface. It is always **up** and **reachable** as long as the device is running.
 - **Analogy:** It's the router's **permanent, stable identity card or license plate**. In BGP, it's used as the **Router ID** because it never goes down.
- **LO Interface (Loopback Interface):**
 - **Definition:** A **virtual network interface** that exists entirely within the operating system. It is not connected to any physical hardware (cable).
 - **Analogy:** It's a **virtual port or internal mirror** where any data sent to it is immediately reflected back to the operating system.

BGP/FRR: VM2 "advertises" the networks it knows about. VM1 and VM3 "learn" the paths automatically. If a new network is added to VM2, it automatically tells the other VMs without you running a single command on them.

Step 3.1: ***multipass shell vm1***

Step 3.2: ***sudo vim /etc/netplan/50-cloud-init.yaml***

The `enp0s2` section should look like this:

```
None
ethernets:
  enp0s2:
    dhcp4: false
    addresses: [172.16.0.10/24]
  enp0s1:
# ...
```

Step 3.3: ***sudo netplan try***

Step 3.4: ***sudo netplan apply***

Step 3.5: ***exit***

Step 3.6: ***multipass shell vm3***

Step 3.7: ***sudo vim /etc/netplan/50-cloud-init.yaml***

The `enp0s2` section should look like this (you can leave `enp0s1` as is):

```
None
ethernets:
    enp0s2:
        dhcp4: false
        addresses: [192.168.0.2/24]
    enp0s1:
# ...
```

Step 3.8: ***sudo netplan try***

Step 3.9: ***sudo netplan apply***

Step 3.10: ***exit***

The next crucial verification is confirming that **IP forwarding** is active on **VM2** (your router), as this is essential for BGP to successfully move traffic between the two different networks.

Step 3.11: ***multipass exec vm2 -- cat /proc/sys/net/ipv4/ip_forward***

Expected output : 1 IP forwarding is currently enabled.

What it does: **Checks the live forwarding status.** It reads a specific file in the Linux kernel to see if packet forwarding is currently active.

Why: We must confirm that **VM2** (our router) is actively allowed to send traffic between the two separate networks (172.16.0.0/24 and 192.168.0.0/24).

Step 3.12: ***multipass exec vm2 -- grep "net.ipv4.ip_forward" /etc/sysctl.conf***

Expected output : net.ipv4.ip_forward=1

The setting is saved and will persist across reboots.

What it does: **Checks the saved forwarding status.** It looks inside the main system configuration file (**sysctl.conf**) for the line that makes the forwarding setting permanent

Why: We need to ensure that if **VM2** reboots, it will **remember** to be a router and keep forwarding enabled.

Step 3.13: ***sudo apt update && sudo apt install -y frr***

Perform this step on all the vms

What it does: **Installs the BGP software**. First, it updates the list of available software, then installs the **Free Range Routing (FRR)** suite.

Why: FRR is the application that allows your VMs to speak the **BGP dynamic routing language**, letting them share network routes automatically.

Step 3.14: ***sudo ip addr add 10.1.1.1/32 dev lo***

What it does: **Assigns the Loopback ID**. It gives the virtual loopback interface (**lo**) of **VM1** the stable, permanent IP address **10.1.1.1**.

Why: This is the **unique, reliable ID** that **VM1** will use as its **BGP Router ID**. It will advertise this address so VM3 can reach it, even if the main network link changes.

Step 3.15: ***sudo ip addr add 10.1.1.2/32 dev lo***

What it does: **Assigns the Loopback ID**. It gives the loopback interface of **VM2** the stable IP address **10.1.1.2**.

Why: This is **VM2's BGP Router ID**. It's the central ID that connects the two ends of the routing topology.

Step 3.16: ***sudo ip addr add 10.1.1.3/32 dev lo***

What it does: **Assigns the Loopback ID**. It gives the loopback interface of **VM3** the stable IP address **10.1.1.3**.

Why: This is the **VM3's BGP Router ID**. This is the final destination network that VM1 will be looking for via BGP.

Step 3.17: ***multipass shell vm1***

Step 3.18: sudo vtysh

What it does: **Launches the FRR Configuration Shell**. It stands for **Virtual Terminal Shell**, which is the specialized command-line interface (CLI) used for configuring and managing the **FRR (Free Range Routing)** suite. **sudo** is used because configuring routing protocols requires root (administrator) privileges.

Why: This is the **control panel** for BGP.

Step 3.19: VM1 Configuration

FRR Command	Explanation
<code>conf t</code>	Enter configuration mode .
<code>router bgp 65001</code>	Starts the BGP process for Autonomous System (AS) 65001 .
<code>bgp router-id 10.1.1.1</code>	Sets the BGP identifier to the stable Loopback IP (10.1.1.1) .
<code>neighbor 172.16.0.20 remote-as 65002</code>	Defines VM2's interface IP (172.16.0.20) as a neighbor and assigns it to AS 65002 .
<code>network 10.1.1.1/32</code>	Instructs BGP to advertise VM1's Loopback IP so that other routers (like VM2) learn how to reach it.
<code>exit</code>	Exit the BGP configuration.

write memory	Saves the configuration so it persists after a reboot.
exit	Exit the vtysh shell.

Step 3.20: *multipass shell vm2*

Step 3.21: *sudo vtysh*

Step 3.22: *VM2 Configuration*

FRR Command	Explanation
conf t	Enter configuration mode .
router bgp 65002	Starts the BGP process for AS 65002 .
bgp router-id 10.1.1.2	Sets the BGP identifier to the stable Loopback IP (10.1.1.2) .
Neighbor 1 (VM1):	
neighbor 172.16.0.10 remote-as 65001	Defines VM1 as the first neighbor, using its interface IP (172.16.0.10).

Neighbor 2 (VM3):	
neighbor 192.168.0.2 remote-as 65003	Defines VM3 as the second neighbor, using its interface IP (192.168.0.2).
network 10.1.1.2/32	Advertises VM2's Loopback IP so it's reachable from both neighbors.
end	Exit back to privileged mode.
write memory	Saves the configuration.
exit	Exit the vtysh shell.

Step 3.23: *multpass shell vm3*

Step 3.24: *sudo vtysh*

Step 3.25: *VM3 Configuration*

FRR Command	Explanation
conf t	Enter configuration mode .

<code>router bgp 65003</code>	Starts the BGP process for AS 65003 .
<code>bgp router-id 10.1.1.3</code>	Sets the BGP identifier to the stable Loopback IP (10.1.1.3) .
<code>neighbor 192.168.0.1 remote-as 65002</code>	Defines VM2's interface IP (192.168.0.1) as the neighbor, using the address VM2 uses for the 192.168.0.0/24 network.
<code>network 10.1.1.3/32</code>	Instructs BGP to advertise VM3's Loopback IP.
<code>end</code>	Exit back to privileged mode (#).
<code>write memory</code>	Saves the configuration.
<code>exit</code>	Exit the <code>vttysh</code> shell.

Step 3.26: *Check BGP Status (On VM2)*

```

multipass shell vm2
sudo vtysh

```

Step 3.27: *show ip bgp summary*

We should see `172.16.0.10` and `192.168.0.2` listed as neighbors. The `State/PfxRcd` column for both should show **Established** (and possibly a number indicating routes received).

Step 3.28: *show ip route*

This command shows the kernel routing table. We should see entries for the loopback addresses of VM1 (10.1.1.1/32) and VM3 (10.1.1.3/32), marked as learned via BGP.

Step 3.29: *multipass exec vm1 -- ping -c 4 10.1.1.3*

Checks if VM1 can reach VM3's stable loopback address across the router (VM2).

Step 3.30: *multipass exec vm3 -- ping -c 4 10.1.1.1*

Checks if VM3 can reach VM1's stable loopback address across the router (VM2).

```
shambhavi.intern@PP-R7MV533L02 ~ % multipass exec vm1 -- ping -c 4 10.1.1.3
PING 10.1.1.3 (10.1.1.3) 56(84) bytes of data.
From 172.16.0.20 icmp_seq=1 Redirect Host(New nexthop: 2.0.168.192)
64 bytes from 10.1.1.3: icmp_seq=1 ttl=63 time=1.00 ms
From 172.16.0.20 icmp_seq=2 Redirect Host(New nexthop: 2.0.168.192)
64 bytes from 10.1.1.3: icmp_seq=2 ttl=64 time=7.20 ms

--- 10.1.1.3 ping statistics ---
2 packets transmitted, 2 received, +2 errors, 0% packet loss, time 1022ms
rtt min/avg/max/mdev = 1.002/4.103/7.204/3.101 ms
shambhavi.intern@PP-R7MV533L02 ~ % multipass exec vm3 -- ping -c 4 10.1.1.1
PING 10.1.1.1 (10.1.1.1) 56(84) bytes of data.
From 192.168.0.1 icmp_seq=1 Redirect Host(New nexthop: 10.0.16.172)
64 bytes from 10.1.1.1: icmp_seq=1 ttl=63 time=0.934 ms
64 bytes from 10.1.1.1: icmp_seq=2 ttl=64 time=7.51 ms
64 bytes from 10.1.1.1: icmp_seq=3 ttl=64 time=3.18 ms

--- 10.1.1.1 ping statistics ---
3 packets transmitted, 3 received, +1 errors, 0% packet loss, time 2086ms
rtt min/avg/max/mdev = 0.934/3.875/7.513/2.730 ms
shambhavi.intern@PP-R7MV533L02 ~ %
```